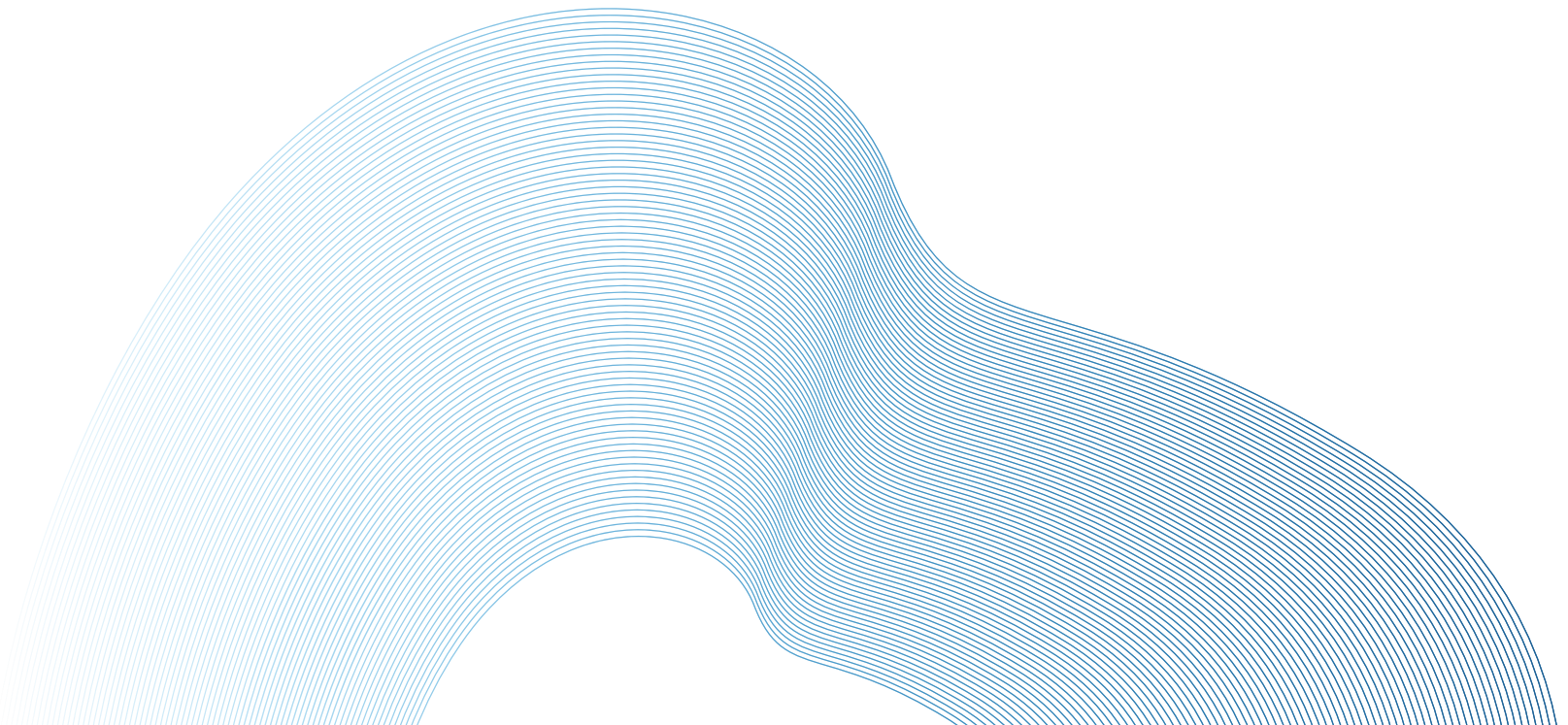
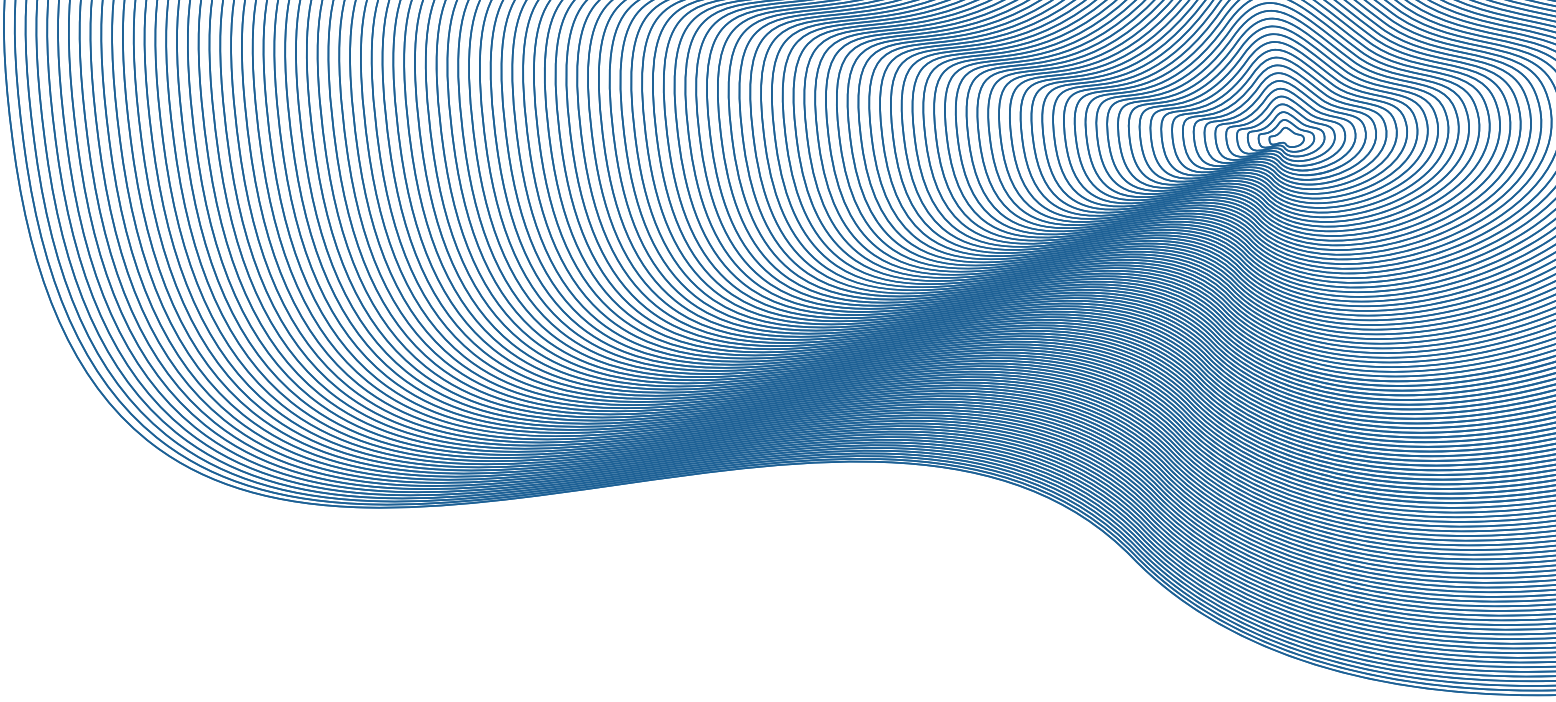
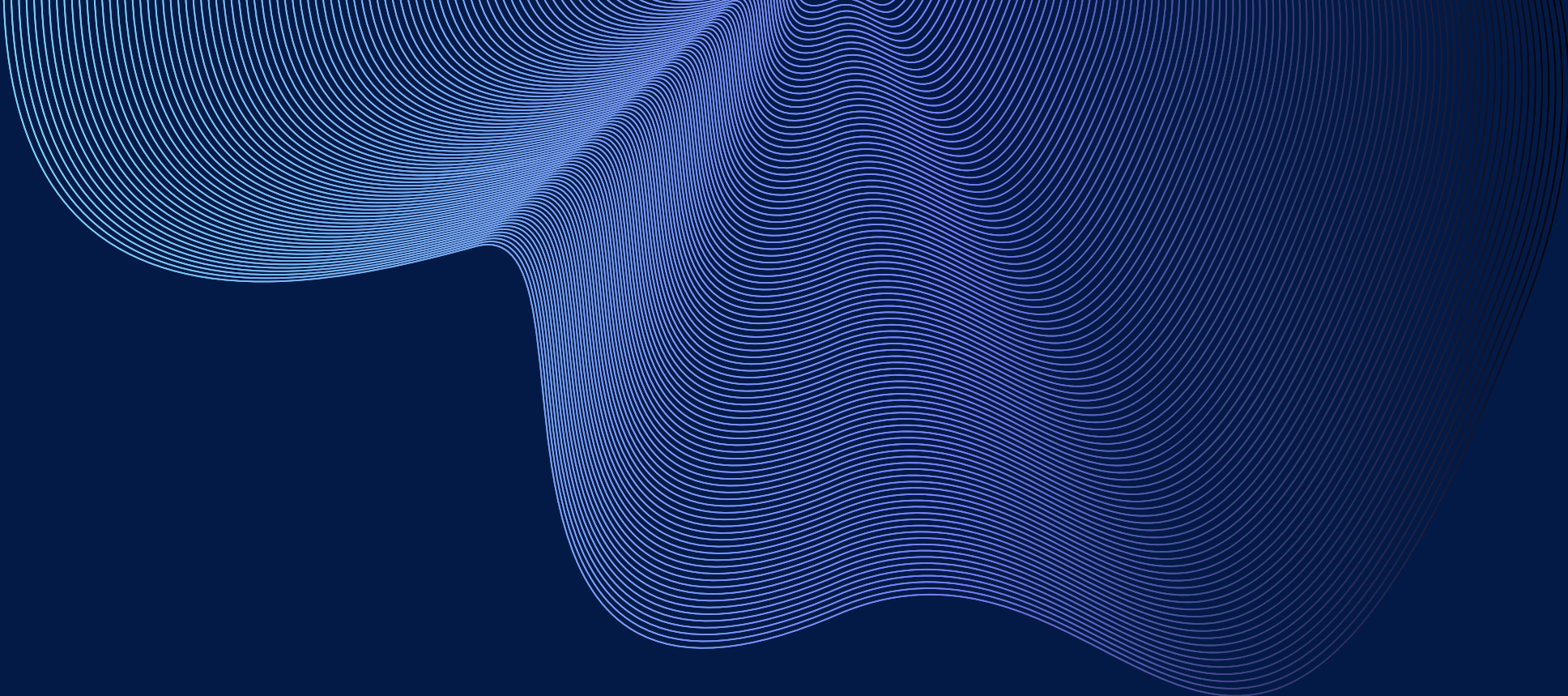
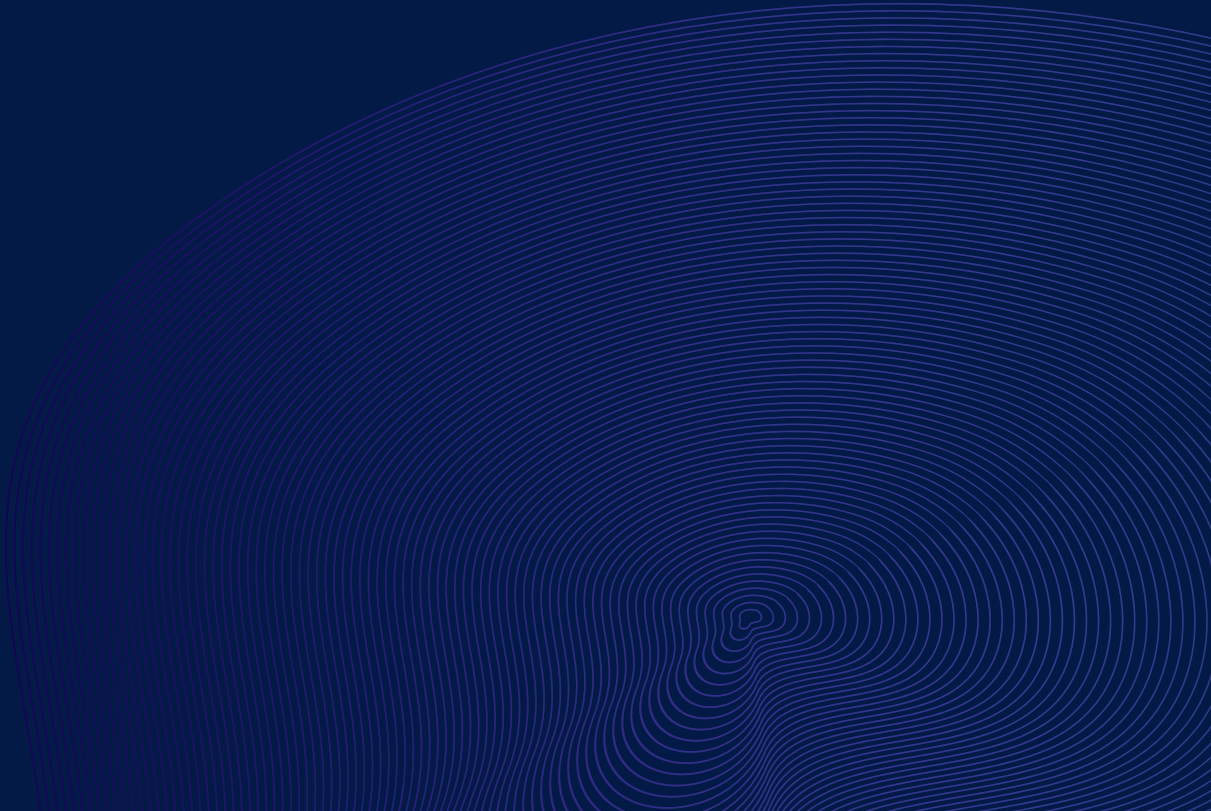




CODE WEB



AULA 06: JAVASCRIPT



OBJETIVOS DA AULA

- O que é e como usar
- Sintaxe básica
- Tipos de dados
- Operadores
- Condições
- Laços
- Funções

O QUE É O JAVASCRIPT E SEU OBJETIVO

É uma linguagem de programação utilizada na criação de scripts para páginas Web, mas não sendo limitada apenas a elas.

Criada em 1995 pelo americano Brendan Eich e implementada no Internet Explorer em 1996.

Utiliza por padrão o modelo *ECMAScript* (*padrão do setor*)

- Universal para todos os navegadores web modernos a partir de 2012.

COMO USAR

Em um arquivo HTML é necessário adicionar a tag `<script>` no fim da tag `<body>`, linkando com "src" o nome do arquivo com final ".js" que será utilizado.

```
1  <!DOCTYPE html>
2  <html lang="en">
3    <head>
4      <meta charset="UTF-8" />
5      <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6      <title>Aula 6</title>
7    </head>
8    <body>
9
10
11      <script src="main.js"></script>
12    </body>
13  </html>
```

OBS: adicionamos a tag `<script>` no fim para garantir que o conteúdo inteiro da página foi carregado antes que um script tenha iniciado.

COMO USAR

É possível colocar código JavaScript diretamente dentro da tag `<script>`, porém essa prática é voltada mais para testes e exemplos.

```
1  <!DOCTYPE html>
2  <html lang="en">
3    <head>
4      <meta charset="UTF-8" />
5      <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6      <title>Aula 6</title>
7    </head>
8    <body>
9      <script src="main.js">
10       | console.log("Hello World!");
11      </script>
12    </body>
13  </html>
```


ALGUMAS PROPRIEDADES DO JAVASCRIPT

A própria engine da linguagem descarta espaços em branco para deixar o código mais leve e rápido de se carregar.

- É possível fazer um arquivo inteiro em apenas 1 linha.

```
1  let exempl = 10;  
2  
3  if(exempl > 5){  
4    |  console.log("Hello World!");  
5  }
```

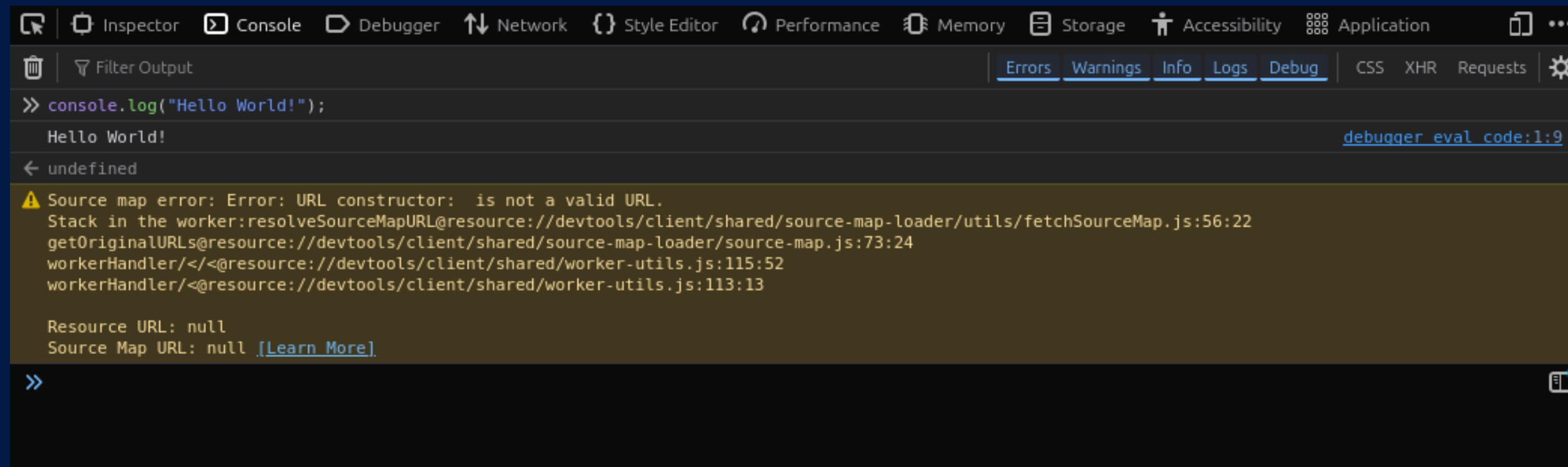
```
1  let exempl = 10; if (exempl > 5) {console.log("Hello World!");}
```

Ambos os códigos são idênticos porém com formatação diferente.

ALGUMAS PROPRIEDADES DO JAVASCRIPT

Para imprimir textos, valores, variáveis, etc usaremos o **console.log**

- printf (praticamente)
- O output se encontra na aba **console** do **Developer Tools** no navegador.



Ignorar a mensagem de warning



VARIÁVEIS

VARIÁVEIS

É uma linguagem dinâmica, logo não é necessário especificar o tipo da variável durante a declaração da mesma.

- Após uma variável for assignada com um tipo, não há problema em assigna-lá com outro tipo posteriormente.

Variáveis indefinidas: foram declaradas, mas não foram inicializadas com um valor.

Variáveis indeclaradas: não chegaram a ser declaradas, são inexistentes.

É possível declarar mais de uma variável utilizando “ , ”

VARIÁVEIS

Usamos **var**, **let** e **const** para declarar variáveis.

Diferenças:

- **let** só podem ser usadas apenas no escopo em que foram declaradas, sendo globais que nem **var**.
- **var** permite redeclarar a variável no mesmo escopo enquanto **let** não permite.

Por conta de boas práticas, iremos priorizar usar **let**.

VARIÁVEIS

```
1  let varUm = 1;
2  var varDois = 2;
3
4  varUm = "João";
5  varDois = 5.64;
6
7  console.log(varUm); // "João"
8  console.log(varDois); // 5.64
```

```
1  if (true) {
2    |  var x = 100;
3    |  let y = 200;
4    |  }
5
6  console.log(x); // 100
7  console.log(y); // ReferenceError: y is not defined
```


VARIÁVEIS

- Usamos a declaração **const** para variáveis cujo valores não serão alterados, nem mesmo pelo programador (read-only).
- É necessário atribuir um valor no momento da declaração.

```
1  const pi = 3.14;
```




TIPOS DE DADOS

TIPOS DE DADOS

operador **typeof**() retorna o tipo de uma variável

unsigned: default para variáveis declaradas mas não inicializadas.

null: Ausência intencional de um valor

number: Representação de números inteiros e reais

- Valores reais que aparecerem como inteiros (ex: 100.00) JS vai converter automaticamente para um valor inteiro.

NaN: Not a Number, representa um número inválido (ex: dividir um caracter por um número)

- Toda operação com NaN retorna NaN
- Não é igual a nenhum valor, nem com ele mesmo
 - `Nan != Nan`
- Comparações com NaN são comparações falhas

TIPOS DE DADOS

string: Representação de textos por meio de ' ' ou " "

- São imutáveis, logo tentar modificar seu conteúdo ocorrerá erro.
- Para modificar uma string é necessário criar outra string de uma já existente

```
1  let string1 = 'teste de';  
2  string1 = string1 + ' string';
```

JavaScript cria uma nova string que contém "teste de string" e apaga as strings que contém "teste de" e " string"

TIPOS DE DADOS

bool: Retorna true ou false, possível converter outros tipos de dados usando Boolean().

Data Type	Values converted to true	Value Converted to false
string	Any non-empty string	"" (empty string)
number	Any Non-zero number	0, NaN
object	Any object	null
undefined	(not relevant)	undefined

```
Boolean(123); // true
Boolean(0); // false
Boolean("texto"); // true
Boolean(""); // false
Boolean(null); // false
Boolean(undefined); // false
Boolean([]); // true
Boolean({}); // true
Boolean(NaN); // false
```


TIPOS DE DADOS

object: coleção de propriedades em pares onde cada uma é definida por meio da propriedade **key-value**

```
1  let pessoa1 = {} // criação de um objeto vazio
2
3  let pessoa2 = {
4      idade: 20,
5      nome: 'zezinho',
6      universidade: { // é possível colocar objetos dentro de outros objetos
7          nomeUni: 'Unioeste',
8          RA: 654321,
9          campus: "cascavel"
10     }
11 }
```

Para acessar propriedades de um objeto há 2 meios:

- **dot notation (.)** = pessoa2.idade
- **array-like notation ([])** = pessoa2['nome']
 - Permite que possam ser usadas em operações, desde que os valores sejam compatíveis



OPERADORES

OPERADORES

- + : Adição de valores ou concatenação caso as duas variáveis sejam strings.
 - No caso de **string + número**, o número é convertido para string e então concatenado. Logo `'10' + 20 = 1020`
- : Subtração de valores, se usado com strings os valores serão convertidos para números através da função `Number()`

```
1 console.log("10" - "5"); // 5
2 console.log("10" - 2); // 8
3 console.log("abc" - 1); // NaN
```

* : Multiplicação de valores

/ : Divisão de valores

%: Resto após realizar uma divisão entre dois valores, dividendo % divisor

- A propriedade da subtração para quando se usa strings vale também para os três

OPERADORES

= : Atribuição, com ele podem ser realizados os **operadores unários**:

- postfix: x-- e x++, depois de se avaliar a passagem
- prefix: --x e ++x, antes de se avaliar a passagem
- Conversão para positivo (+x) e negativo (-x)

Suporta a forma que usa a propria variável na operação: +=, -=, *=, /=, %= e **=

- Atribuição de comparação: >, <, >=, <=, ==, != (conhecidos)
- === e !== : estritamente igual (ou diferente), JS não irá converter valores automaticamente

```
console.log("10" == 10); // true  
console.log("10" === 10); // false
```

OBS: troque por !== e o retorno será true

OPERADORES

Comparações com strings

Será comparado caracter por caracter de forma numérica (ASCII), podendo gerar resultados inesperado:

```
1  let f1 = 'apple',  
2  |     f2 = 'Banana';  
3  let result = f2 < f1;  
4  console.log(result); // true
```

Lembre-se da conversão realizada quando o valor não é numérico

Cuidado! "2" < "10" retorna **false** por conta de "2" ser comparado com "1" e não com "10"

Assim como, "abc" < 5 também retorna **false**, já que "abc" vira NaN, resultando em uma comparação falha

OPERADORES

Operadores lógicos

!: NOT

- `!a; == false`
- `!!a; == true`

&& : AND

- `a && b;`

|| : OR

- `a || b;`

?? : Nullish Coalescing, retorna o primeiro valor que não é NaN ou undefined

- `a ?? b;`
- Não trata 0, false, ou `""` como valores falsos



CONDIÇÕES

CONDIÇÕES

Operações que tomam decisões baseadas em condições de valores **bool**

if, else, e else if:

```
if(condição1){  
    //se atender a condição  
    // valor bool = 1  
    //executa o que tiver aqui  
} else if (condição2){  
    // caso a condição anterior deu false (0)  
    // verifica o valor bool dessa condição  
    // se sim executa  
} else {  
    // senao  
    // executa o que tiver aqui  
}
```

Operador ternário: versão reduzida do if

- *condição ? seTrueExecutaIsso : seFalseExecutaEste;*

CONDIÇÕES

switch case: Utiliza de === em seus cases

```
let fruta = "maçã";

switch (fruta) {
  case "banana":
    console.log("é banana");
    break;
  case "maçã":
    console.log("é maçã");
    break;
  case "laranja":
    console.log("é laranja");
    break;
  default:
    console.log("fruta desconhecida");
}
```


CONDIÇÕES

comma operator: Pega duas expressões, calcula as expressões da esquerda para direita e retorna o valor da expressão direita

```
let x = 10;  
let y = (x++, x + 1);  
console.log(x, y); // 11 12  
  
//versão compacta de:  
let x = 10;  
x++;  
let y = x + 1;  
console.log(x, y);
```

Muito utilizado em for loops para atualizar múltiplas variáveis de uma vez



LAÇOS DE REPETIÇÃO

LAÇOS DE REPETIÇÃO

while: Executa um bloco de código enquanto uma condição for verdadeira

```
let cont = 1;

while (cont <= 5) {
  console.log(cont);
  cont++;
}
```

Iteração de strings em uma array:

```
let nomes = ['João', 'Gabriel', 'Maria', 'Zezinho'];
let i = 0;
while (i < nomes.length) {
  // nao colocar mais de dois valores dentro de um ${}
  console.log(`${i + 1}.${nomes[i]}*`);
  i++;
}
```


LAÇOS DE REPETIÇÃO

for loop: Útil quando se sabe exatamente o número de vezes que algo deve se repetir

```
for (let i = 1; i <= 5; i++) {  
  console.log("número = ", i);  
}
```

let i = 1 é a inicialização do iterador, executado apenas 1 vez quando o for inicia

i <= 5 é a condição de parada

i++ é o incrementador, executado uma vez no final de cada iteração

LAÇOS DE REPETIÇÃO

break: termina o loop de um laço de repetição

continue: termina a execução da iteração atual mas não termina o loop, apenas pula para a parte de incrementação para rodar a próxima iteração

Úteis para quando se tem condições específicas que se quer evitar e servir para condições de paradas, principalmente quando se tem laços de repetição dentro de outros laços



FUNÇÕES

FUNÇÕES

Utilizadas para deixar o código mais compacto e reutilizável

Declaradas a partir da keyword **function**

```
function nomeFuncao(parametro1, parametro2){ // caso nao tenha parametros, deixar vazio  
    // corpo da função  
}
```


FUNÇÕES

Chamamento de funções:

nomeFuncao(argumento1, argumento2);

- A ordem dos argumentos importa com a ordem dos parâmetros

Para a função retornar um valor, é preciso especificar utilizando o keyword **return** seguido do valor ou expressão desejada

```
function exemp1(a, b){  
  let r = a + b;  
  return r; // == return a + b;  
}
```


FUNÇÕES

Objeto **arguments** representa os argumetos passados na função, podendo usar **[]** para acessar os argumentos em sua devida posição e **length** para pegar a quantidade dos mesmos

```
function somatorio() {  
    let soma = 0;  
    for (let i = 0; i < arguments.length; i++) {  
        soma += arguments[i];  
    }  
    return soma;  
}  
  
x = somatorio(1, 1, 1, 1);
```




INTRODUÇÃO AO DOM

DOM (DOCUMENT OBJECT MODEL)

- Uma página Web é um documento que pode ser exibido na janela do navegador ou como fonte HTML
- DOM representa esse documento usando uma série de objetos para que assim possa alterá-lo através de uma linguagem script como o JavaScript
- Permite deixar um documento HTML mais dinâmico e iterativo
- Funciona por meio da comunicação com o navegador por meio de propriedades, métodos e eventos
- Possui padrões W3C DOM e WHATWG DOM implementados na maioria dos navegadores atualmente

DOM (DOCUMENT OBJECT MODEL)

Exemplo simples:

```
let nome1 = document.getElementById("strg");
```

nome1 referência o elemento “strg” do documento HTML, pego através de seu elemento de id único que lhe foi atribuído

Será melhor estudado na próxima aula



OBRIGADO!